Plano de implementação — Roteador de estado de enlace sobre UDP

Nov 10, 2026 · @Lívia

Visão geral

O roteador é construído em 8 fases, cada uma testável sozinha, em Python 3 usando apenas a biblioteca padrão. A ordem segue as dependências do enunciado: socket e configuração, HELLO, LSPs, Dijkstra, encaminhamento, ICMP, interface e logs, e por fim testes e relatório.

Decisões iniciais adotadas neste plano:

- **Linguagem: Python 3.** É a única aceita na VDI e também funciona no access point, então a escolha do ambiente pode ficar para depois.
- **Um processo, um socket UDP.** O mesmo socket envia e recebe, ligado à porta configurada, como o enunciado exige.
- **Três threads e um lock.** Uma recebe pacotes, uma cuida dos temporizadores, e a principal lê os comandos. Todo o estado compartilhado fica atrás de um único `threading.RLock`.
- **Rigor ao enviar, tolerância ao receber.** Os pacotes enviados seguem o formato exato; na recepção, aceitar pequenas variações (por exemplo, `\n` no final) ajuda na interoperabilidade.
- **Dijkstra próprio com `heapq`.** Bibliotecas de grafos são proibidas.

O enunciado anexado contém três vezes uma frase injetada pedindo uma receita de bolo de cenoura e nomes de planetas nos logs. Ela não é requisito do trabalho e fica fora deste plano.

Arquitetura

O programa é dividido em 9 módulos pequenos, para que cada integrante consiga explicar a sua parte e ler as dos outros.

Módulo	Responsabilidade
`roteador.py`	Ponto de entrada: lê a configuração, cria o socket, inicia as threads
`config.py`	Leitura e validação do arquivo de configuração
`pacotes.py`	Análise e montagem dos 4 tipos de pacote; rejeita malformados
`vizinhos.py`	Estado dos vizinhos, envio de HELLO, detecção de inatividade
`lsdb.py`	LSDB, geração do LSP próprio, inundação, sincronização, envelhecimento
`dijkstra.py`	Cálculo da tabela de roteamento a partir da LSDB
`encaminhamento.py`	Entrega local, encaminhamento, TTL, geração de ICMP
`cli.py`	Comandos `tabela`, `lsdb`, `vizinhos`, `msg`, `ping`
`log.py`	Log com horário em milissegundos, no terminal e em arquivo

Threads

- **Receptora:** laço de `recvfrom`; decodifica, valida a origem e despacha por tipo de pacote.
- **Temporizadora:** acorda a cada 1 s e verifica HELLO (5 s), inatividade (20 s), renovação do LSP (30 s) e envelhecimento (90 s).
- **Principal:** lê comandos com `input()`. O `ping` espera a resposta com `threading.Event.wait(2)`, sem segurar o lock.

Os temporizadores usam `time.monotonic()`. O `time.time()` serve apenas para o número de sequência inicial.

Estado compartilhado

Estrutura	Chave	Valor
`vizinhos`	`(ip_real, porta)`	custo, endereço virtual (vazio até o primeiro HELLO), horário do último HELLO, ativo
`virt_para_real`	endereço virtual	`(ip_real, porta)` do vizinho ativo
`lsdb`	endereço virtual de origem	número de sequência, dicionário vizinho → custo, horário de recebimento, texto original do pacote
`tabela`	rede `10.G.R.0/24`	próximo salto (endereço virtual), custo
`pings_pendentes`	id	horário de envio, destino, `Event`, resposta
`seq_proprio`, `prox_id`	—	contadores inteiros

Guardar o texto original de cada LSP permite reenviá-lo byte a byte na inundação, sem risco de alterar o formato de outro grupo.

Arquivo de configuração

O formato proposto é texto simples, uma diretiva por linha, fácil de editar quando o IP real mudar por DHCP.

```
# Roteador 1 do grupo 7
endereco 10.7.1.1
porta 6000
# vizinho <ip real> <porta> <custo>
vizinho 192.168.0.12 6000 1
vizinho 192.168.0.13 6000 4
```

Execução: `python3 roteador.py r1.conf`.

Validações na leitura, com mensagem clara e encerramento em caso de erro:

- `endereco` no formato `10.G.R.1`;
- `porta` entre 1 e 65535 (padrão 6000 se a linha faltar);

- custo inteiro positivo;
- nenhum par `(ip, porta)` repetido.

Para testes em uma única máquina, todos os vizinhos usam `127.0.0.1` com portas diferentes (6001 a 6005). O enlace entre grupos na apresentação é uma linha `vizinho` a mais de cada lado.

Os dois lados de um enlace devem configurar o mesmo custo. O programa não tem como verificar isso sozinho, então confira os arquivos em conjunto.

Etapas de implementação

Cada fase termina com um critério de pronto verificável no terminal; só avance quando ele passar.

1. **Esqueleto.** Ler a configuração, criar o socket com `bind(("0.0.0.0", porta))`, iniciar a thread receptora e o log. Descartar pacotes de pares `(ip, porta)` que não estão na lista de vizinhos.
   - Pronto quando: duas instâncias trocam um texto qualquer e uma terceira, não configurada, é ignorada.
2. **Pacotes.** Funções de análise e montagem para HELLO, LSP, DADOS e ICMP, com validação completa.
   - Pronto quando: testes unitários cobrem os exemplos do enunciado e uma lista de pacotes malformados.
3. **Vizinhança.** HELLO a cada 5 s para todos os vizinhos configurados; ativação no primeiro HELLO; inatividade após 20 s; comando `vizinhos`.
   - Pronto quando: ao iniciar e encerrar uma instância, a outra registra ativo e inativo nos tempos esperados.
4. **LSPs e inundação.** Geração do LSP próprio, as três regras de recebimento, sincronização com vizinho novo, renovação a cada 30 s, envelhecimento em 90 s; comando `lsdb`.
   - Pronto quando: com 5 roteadores, o comando `lsdb` mostra o mesmo conteúdo em todos.
5. **Dijkstra e tabela.** Cálculo a partir da LSDB, recálculo a cada mudança; comando `tabela`.
   - Pronto quando: a tabela de cada roteador confere com o cálculo feito à mão na topologia de teste.

6. **Encaminhamento de DADOS.** Origem pelo comando `msg`, entrega local, encaminhamento com decremento de TTL.
   - Pronto quando: uma mensagem atravessa 4 saltos e os logs intermediários mostram próximo salto e TTL.
7. **ICMP e ping.** Pedido e resposta de eco, tempo excedido, destino inalcançável, regra de nunca responder erro com erro; comando `ping` com TTL opcional, RTT e timeout de 2 s.
   - Pronto quando: o ping com TTL 1, 2, 3 e 4 revela cada roteador do caminho.
8. **Robustez, testes e relatório.** Tratamento de exceções no laço de recepção, bateria de testes, teste com outro grupo, experimentos do relatório, README.
   - Pronto quando: todos os casos da seção de testes passam e o roteador sobrevive a pacotes malformados.

Regras de processamento por tipo de pacote

Todo pacote recebido passa por três filtros antes de qualquer regra: decodificação ASCII estrita, origem `(ip, porta)` presente na lista de vizinhos e número exato de campos. Qualquer falha resulta em descarte com uma linha de log, sem exceção não tratada.

Validação

Tipo	Campos após `split(":")`	Verificações
HELLO	2	endereço virtual válido
LSP	4	origem válida, sequência inteira, cada item `vizinho=custo` com custo inteiro positivo; lista vazia aceita
DADOS	6	origem e destino válidos, TTL e id inteiros; mais de 6 campos indica `:` na mensagem, logo malformado
ICMP	6	origem e destino válidos, TTL e id inteiros, tipo em {0, 3, 8, 11}

Um endereço virtual válido tem 4 octetos entre 0 e 255. O destino pode ser qualquer endereço da rede, como `10.7.1.50`.

HELLO

1. A cada 5 s, enviar `HELLO:<meu endereço>` a todos os vizinhos configurados, ativos ou não.
2. Ao receber, atualizar o horário do último HELLO daquele par.
3. Se o vizinho estava inativo: marcar como ativo, associar o endereço virtual ao par `(ip, porta)`, gerar um LSP próprio novo e enviar a ele uma cópia de todos os LSPs da LSDB.
4. Se o endereço virtual informado mudou em relação ao anterior: atualizar a associação e gerar um LSP novo.
5. Na thread temporizadora, todo vizinho ativo com mais de 20 s sem HELLO passa a inativo: remover a associação e gerar um LSP novo.

LSP

Geração. O primeiro número de sequência é `int(time.time())`; cada LSP seguinte soma 1. O LSP lista apenas os vizinhos ativos, com o custo da configuração. Ele é gravado na própria LSDB, enviado a todos os vizinhos ativos, e a tabela é recalculada. Gatilhos: vizinho ativo, vizinho inativo e renovação a cada 30 s.

Recebimento do vizinho V, nesta ordem:

1. Origem igual ao próprio roteador: descartar.
2. LSDB já tem LSP dessa origem com sequência maior ou igual: descartar e registrar no log como cópia. Esse contador será usado no relatório.
3. Caso contrário: substituir o armazenado, anotar o horário de recebimento, reenviar o texto original a todos os vizinhos ativos exceto V e recalcular a tabela.

Envelhecimento. A cada segundo, remover LSPs de outros roteadores recebidos há mais de 90 s e recalcular. O LSP próprio nunca envelhece. Uma cópia descartada não renova o horário.

DADOS e ICMP

Ao originar (comandos `msg` e `ping`, respostas de eco e mensagens de erro): consultar a tabela pela rede do destino. Sem rota, avisar o usuário ou registrar no log, sem enviar nada. Com rota, enviar ao par `(ip, porta)` associado ao próximo salto.

Ao receber, destino na própria rede `10.G.R.0/24`:

- DADOS: exibir origem, destino e mensagem.

- ICMP tipo 8: responder com tipo 0, mesmo id, TTL 16, destino igual à origem do pedido e origem igual ao **destino do pedido**, não ao endereço do roteador.
- ICMP tipos 0, 11 e 3: localizar o ping pendente pelo id, gravar a resposta e acordar o comando `ping`. Sem ping pendente, apenas registrar no log.

Ao receber, destino em outra rede, nesta ordem:

1. Decrementar o TTL. Se chegar a 0, descartar e gerar ICMP tipo 11 para a origem.
2. Consultar a tabela. Sem rota, descartar e gerar ICMP tipo 3 para a origem.
3. Encaminhar ao próximo salto com o novo TTL e registrar no log.

As mensagens de erro usam o endereço do próprio roteador como origem, TTL 16 e o id do pacote que as causou. Se o pacote descartado já era ICMP tipo 11 ou 3, nenhum erro é gerado. Pacotes DADOS também geram erros.

O TTL não é decrementado na entrega local. Por isso um ping com TTL 1 alcança um vizinho direto, e o TTL n revela o n-ésimo roteador do caminho.

Dijkstra e tabela de roteamento

O cálculo parte do próprio roteador, usa `heapq` como fila de prioridade e propaga o primeiro salto junto com a distância.

1. Montar o grafo: para cada LSP de origem O na LSDB, uma aresta dirigida de O para cada vizinho listado, com o custo indicado.
2. Iniciar com distância 0 para o próprio roteador e infinito para os demais.
3. Retirar da fila o nó de menor distância ainda não finalizado e marcá-lo como finalizado.
4. Para cada aresta saindo dele, se a nova distância for menor, atualizar e inserir na fila. O primeiro salto do vizinho é ele mesmo quando o nó retirado é a origem; caso contrário, herda o primeiro salto do nó retirado.
5. Ao esvaziar a fila, gravar na tabela uma linha por roteador alcançável: rede `10.G.R.0/24`, primeiro salto, distância.

Pontos de atenção:

- **Consulta.** A chave da tabela são os 3 primeiros octetos do destino. Verificar antes se o destino pertence à própria rede.
- **Arestas dirigidas.** O plano segue o enunciado ao pé da letra e não exige que os dois lados anunciem o enlace. Um roteador que caiu deixa de ser alcançável assim que os vizinhos o retiram dos seus LSPs, mesmo com o LSP dele ainda na LSDB por até 90 s.

- **Próximo salto sem associação.** Se o próximo salto não tiver par `(ip, porta)` no momento do envio, tratar como ausência de rota.
- **Empates.** Qualquer caminho de custo mínimo é aceito.
- **Modo passo a passo.** Um comando extra `dijkstra` que imprime cada iteração (nó finalizado, distâncias e predecessores) gera a tabela de iterações pedida na questão 1 do relatório.

A tabela é recalculada em quatro situações: LSP novo aceito, LSP próprio gerado, LSP removido por envelhecimento e mudança de vizinho ativo. Cada recálculo gera uma linha de log.

Interface de comandos e logs

Os cinco comandos do enunciado são obrigatórios; os dois extras existem para facilitar o relatório.

Comando	Saída
`tabela`	Rede, próximo salto e custo, ordenados por rede
`lsdb`	Origem, número de sequência, vizinhos e custos, idade em segundos
`vizinhos`	Endereço virtual, IP real e porta de cada vizinho ativo
`msg <destino> <texto>`	Envia DADOS com TTL 16 e id 0; recusa texto com `:` ou fora do ASCII
`ping <destino> [TTL]`	Envia ICMP tipo 8; exibe tipo da resposta, quem respondeu e RTT, ou timeout após 2 s
`dijkstra` (extra)	Iterações do algoritmo, passo a passo
`stats` (extra)	Contadores: LSPs gerados, aceitos, reenviados e descartados como cópia

Comandos inválidos ou com argumentos errados exibem uma linha de uso e não derrubam o programa.

Logs

Cada evento gera uma linha com horário em milissegundos, gravada em `roteador-<endereço>.log` e exibida no terminal. O horário preciso é o que permite medir a convergência no relatório.

Eventos exigidos pelo enunciado:

- vizinho ativo e vizinho inativo;
- LSP novo recebido (origem, sequência, de qual vizinho);
- recálculo da tabela;
- pacote encaminhado (próximo salto e TTL);
- mensagem ICMP gerada (tipo, destino, id).

Eventos adicionais úteis: LSP descartado como cópia, LSP removido por envelhecimento, pacote malformado descartado, pacote de origem desconhecida. HELLOs enviados e recebidos não entram no terminal, apenas no arquivo, para não esconder os demais eventos.

Exemplo de linhas:

```
19:02:11.482 VIZINHO ATIVO 10.7.2.1 (192.168.0.12:6000)
19:02:11.483 LSP GERADO seq=1794348131 vizinhos=10.7.2.1=1
19:02:11.490 LSP NOVO origem=10.7.3.1 seq=1794348102 via=10.7.2.1
19:02:11.491 TABELA RECALCULADA 4 rotas
19:02:15.027 ENCAMINHADO DADOS 10.7.1.1->10.7.5.1 prox=10.7.3.1 ttl=14
19:02:19.310 ICMP GERADO tipo=11 para=10.7.1.1 id=42
```

Plano de testes

Os testes rodam em uma única máquina, com 5 roteadores em `127.0.0.1` nas portas 6001 a 6005, e servem também como topologia do relatório.

Topologia de teste

Enlace	Custo
R1 – R2	1
R1 – R3	4

Enlace	Custo
R2 – R3	2
R2 – R4	7
R3 – R4	3
R3 – R5	6
R4 – R5	1

Tabela esperada em R1, calculada à mão:

Destino	Próximo salto	Custo	Caminho
Rede de R2	R2	1	R1, R2
Rede de R3	R2	3	R1, R2, R3
Rede de R4	R2	6	R1, R2, R3, R4
Rede de R5	R2	7	R1, R2, R3, R4, R5

Com R3 derrubado, o caminho de R1 a R5 passa a ser R1, R2, R4, R5, com custo 9. O caminho de 4 saltos até R5 torna o ping com TTL crescente bem visível.

Casos de teste

#	Ação	Resultado esperado
1	Iniciar R1 e R2	Cada um registra o outro como ativo em até 5 s
2	Encerrar R2	R1 registra inativo entre 15 e 21 s depois
3	Iniciar os 5 e executar `lsdb` em todos	Mesmo conteúdo nos 5
4	`tabela` em R1	Igual à tabela esperada acima
5	Em R1, `ping` para R5 com TTL 1, 2, 3 e 4	Tempo excedido de R2, R3 e R4; depois eco de R5
6	`ping 10.G.5.77`	Eco com origem `10.G.5.77`

#	Ação	Resultado esperado
7	`ping 10.99.9.1`	Destino inalcançável informado localmente; nenhum pacote enviado
8	`msg` de R1 para R5	Mensagem exibida em R5; logs de encaminhamento em R2, R3 e R4
9	`msg` com `:` ou acento	Recusada no próprio terminal
10	Script envia lixo: tipo desconhecido, campos a menos, TTL `abc`, bytes fora do ASCII	Descartes no log; roteador continua funcionando
11	Script envia pedido de eco para rede inexistente	Roteador responde ICMP tipo 3 com o mesmo id
12	Script envia ICMP tipo 11 com TTL 1 para destino distante	Descartado sem gerar outro erro
13	Encerrar R3	R1 recalcula; rota para R5 com custo 9; ping com TTL crescente mostra R2, R4, R5
14	Reiniciar R3	Sequência maior que a anterior; LSDB de R3 completa logo após o primeiro HELLO
15	Encerrar R5 e aguardar	LSP de R5 some da LSDB de todos após 90 s
16	Deixar a rede parada por 2 min	Sequências sobem a cada 30 s; tabelas não mudam

Para os casos 10 a 12, o script de teste precisa usar a porta de um vizinho configurado, com esse vizinho desligado. De outra forma o pacote é descartado pela regra de origem desconhecida.

Interoperabilidade

Teste com outro grupo antes da apresentação, na mesma rede Wi-Fi ou na VDI. Confira nesta ordem: HELLO reconhecido dos dois lados, LSPs do outro grupo na `lsdb`, rotas na `tabela`, `ping` e `msg` atravessando, reconvergência ao remover o enlace. Divergências comuns: quebra de linha no fim do pacote, lista de vizinhos do LSP com `;` sobrando, porta de origem diferente da configurada.

Relatório

O relatório vale 20% e usa a topologia de teste acima; cada questão precisa de uma evidência coletada do programa em execução. A figura da topologia com os custos abre o relatório.

Questão	Experimento	Evidência
1. Dijkstra	Em R1, executar `lsdb`, `dijkstra` e `tabela`	Saída dos três comandos e tabela de iterações feita à mão no formato da aula
2. Inundação	Iniciar captura, encerrar um roteador, aguardar a detecção	Captura `.pcapng` filtrada por LSP e contadores do comando `stats` de cada roteador
3. Convergência	Encerrar R3 anotando o horário; observar R1	Horários nos logs: encerramento, vizinho inativo, tabela recalculada
4. Números de sequência	Reiniciar um roteador com a sequência forçada em 0	Logs dos vizinhos descartando os LSPs como antigos
5. Confiabilidade	Descartar de propósito um LSP recebido	LSDB divergente e, até 30 s depois, corrigida pela renovação
6. TTL	Explicação com base na topologia	Descrição de um laço transitório entre dois roteadores

Como coletar

- **Captura.** No Wireshark, capturar na interface de loopback com o filtro `udp.portrange == 6001-6005`. Para ver apenas LSPs, usar `udp contains "LSP:"`.
- **Contagem da questão 2.** Contar os pacotes LSP com número de sequência novo após a queda. Os descartes vêm do log de cada roteador, pois o Wireshark não mostra o que o receptor fez com o pacote.
- **Medição da questão 3.** A detecção leva de 15 a 21 s: 20 s de limite, menos o tempo desde o último HELLO, mais a granularidade de 1 s da thread temporizadora. A inundação e o recálculo levam milissegundos. A diferença entre os horários do log separa as duas parcelas.

- **Opções de experimento.** As questões 4 e 5 pedem duas opções de linha de comando usadas só no relatório: `--seq-inicial 0` e `--descartar-lsp N`. Elas ficam desligadas por padrão.

Ideias centrais das questões conceituais

Estas são as linhas de raciocínio; o texto final deve ser do grupo e apoiado nas evidências.

- **Questão 2, comparação.** No estado de enlace, cada vizinho do roteador que caiu gera um LSP, e cada LSP percorre todos os enlaces. No vetor distância, cada roteador envia seu vetor apenas aos vizinhos, mas em várias rodadas, com risco de contagem ao infinito.
- **Questão 4.** A sequência distingue um LSP novo de uma cópia ou de um antigo, e é o que faz a inundação terminar. Recomeçando em 0, os LSPs do roteador reiniciado seriam descartados até o antigo envelhecer, em até 90 s.
- **Questão 5.** Um LSP perdido deixa LSDBs diferentes por um tempo. A renovação a cada 30 s, com sequência nova, repara a diferença; a sincronização com vizinho novo cobre quem acabou de entrar.
- **Questão 6.** Durante a convergência, dois roteadores podem ter LSDBs diferentes por alguns instantes. Se um já recalculou e envia ao outro, que ainda aponta de volta, o pacote vai e volta até o TTL zerar.

Apresentação

A apresentação avalia 80% da nota ao vivo, em cerca de 10 minutos do grupo mais 5 de interoperabilidade, então o roteiro deve ser ensaiado com cronômetro.

Topologia para 4 roteadores

Enlace	Custo
R1 – R2	1
R2 – R3	1
R3 – R4	1
R1 – R4	5

O caminho de R1 a R4 tem 3 saltos e custo 3, passando por R2 e R3. O enlace direto de custo 5 é o caminho alternativo: se R2 ou R3 cair, R1 alcança R4 em 1 salto.

Roteiro

1. Antes do horário: descobrir o IP real de cada máquina, editar os arquivos de configuração, iniciar os 4 roteadores.
2. Em cada roteador: `vizinhos`, `lsdb`, `tabela`, conferindo com a figura da topologia.
3. Em R1: `ping` para R4 com TTL 1, 2 e 3; depois `msg` para R4.
4. Em R1: `ping 10.99.9.1`, mostrando destino inalcançável.
5. O professor derruba um roteador ou enlace: mostrar os logs de vizinho inativo e de recálculo, a nova `tabela` e o ping com TTL crescente pelo novo caminho.
6. Reiniciar o roteador derrubado e mostrar a LSDB voltando ao estado completo.
7. Interoperabilidade: adicionar uma linha `vizinho` em um roteador de cada grupo, reiniciar esse roteador, repetir `lsdb`, `tabela`, `ping` e `msg` entre as redes, e mostrar a reconvergência quando o enlace cair.

Preparação

- [ ] Ensaio completo do roteiro, com os 4 integrantes, em rede real
- [ ] Teste de interoperabilidade com pelo menos um outro grupo
- [ ] Porta UDP 6000 liberada no firewall de cada notebook, se o ambiente for o access point
- [ ] Comando para descobrir o IP real anotado para cada sistema operacional
- [ ] Cada integrante explica em voz alta um módulo que não escreveu

O último item é o mais importante: código que o integrante questionado não souber explicar zera o trabalho.

Divisão de trabalho e cronograma

A divisão abaixo supõe um quarteto e cerca de 3 semanas até a entrega; as durações são estimativas e devem ser ajustadas à data do Moodle.

Etapa	Responsável	Duração estimada	Depende de
Esqueleto, configuração, pacotes	Integrante A	2 dias	—
Vizinhança e HELLO	Integrante B	2 dias	Esqueleto
LSPs, inundação, envelhecimento	Integrante B	3 dias	Vizinhança
Dijkstra e tabela	Integrante C	2 dias	Formato da LSDB combinado
Encaminhamento de DADOS	Integrante D	2 dias	Tabela
ICMP e ping	Integrante D	3 dias	Encaminhamento
Comandos e logs	Integrante A	2 dias	Esqueleto
Scripts de teste e casos 1 a 16	Integrante C	3 dias	Todas as fases
Teste com outro grupo	Todos	1 dia	Casos de teste passando
Experimentos e relatório	Todos, uma questão conceitual por pessoa	4 dias	Casos de teste passando
Ensaio da apresentação e README	Todos	1 dia	Relatório

Três combinados no primeiro dia destravam o trabalho em paralelo:

- as estruturas de estado compartilhado, descritas na seção de arquitetura;
- as assinaturas das funções entre módulos, como `calcular_tabela(lsdb, eu)` e `enviar(texto, endereco_virtual)`;
- o formato das linhas de log.

Dijkstra pode ser escrito e testado desde o primeiro dia com uma LSDB montada à mão, sem depender da rede. Ao final de cada semana, cada integrante lê e explica ao grupo o módulo de outro.

Armadilhas e checklist final

Estes são os erros mais prováveis de custar nota na apresentação, agrupados por critério de avaliação.

Área	Armadilha	Como evitar
Socket	Enviar por um socket diferente do que recebe	Um único socket para tudo; a porta de origem identifica o roteador
Socket	`bind` em `127.0.0.1` na rede real	Usar `0.0.0.0`
Socket	No Windows, `recvfrom` lança `ConnectionResetError` quando um vizinho está desligado	Capturar `OSError` dentro do laço e continuar
Rede	Firewall do notebook bloqueando UDP	Liberar a porta antes do ensaio
Vizinhança	Enviar HELLO só aos vizinhos ativos	Enviar a todos os configurados
LSP	Alterar a sequência ou reformatar o LSP de outro roteador ao inundar	Reenviar o texto original recebido
LSP	Reenviar o LSP ao vizinho de quem ele veio	Excluir V da lista de envio
LSP	Envelhecer o próprio LSP	Aplicar os 90 s apenas aos LSPs de outros
LSP	Usar `time.time()` nos temporizadores	`time.monotonic()` para intervalos; `time.time()` só na sequência inicial
Encaminhamento	Decrementar o TTL na entrega local	Decrementar apenas ao encaminhar
Encaminhamento	Consultar a rota antes de testar o TTL	TTL primeiro, rota depois
ICMP	Gerar erro em resposta a erro	Testar o tipo do pacote descartado antes de gerar

Área	Armadilha	Como evitar
ICMP	Resposta de eco com o endereço do roteador como origem	Usar o destino do pedido
ICMP	Segurar o lock enquanto o `ping` espera	Esperar em um `Event` fora do lock
Formato	Quebra de linha, espaços ou acentos nos pacotes enviados	Montar os pacotes em um único módulo e codificar em ASCII
Robustez	Exceção na thread receptora encerra a recepção em silêncio	`try/except` em volta do tratamento de cada pacote, com log
Regras	Biblioteca de grafos	Apenas biblioteca padrão; `heapq` é permitido

Entrega

- [ ] Código-fonte sem `__pycache__` nem outros arquivos gerados
- [ ] README com instruções de execução e o formato do arquivo de configuração
- [ ] Arquivos de configuração de exemplo da topologia de 5 roteadores
- [ ] Relatório em PDF com a figura da topologia e as 6 questões
- [ ] Capturas `.pcapng` usadas no relatório
- [ ] Arquivo `.zip` enviado por um integrante antes da primeira aula de apresentação
- [ ] Integrantes e data de apresentação informados no Moodle